Fredrik Hammarberg

Optimizing Weak Reference Processing in the JVM Z Garbage Collector

A Performance Analysis and Optimization of the ZGC Reference Processing Pipeline

UPPSALA
UNIVERSITET

Abstract
PLACEHOLDER

Acknowledgments

I would like to express my sincere gratitude to my supervisor for his guidance and support throughout this thesis work. It is through his help, encouragement, and expertise that I was able to learn so much about the JVM and bring this thesis to completion.

I would also like to thank my subject reviewer for his valuable academic guidance and thoughtful feedback, which helped strengthen the structure and quality of this thesis.

Additionally, I would like to thank the Department of Information Technology at Uppsala University for providing the resources necessary to conduct the performance evaluations and experiments described in this thesis.

Lastly, I would also like to thank my colleagues at Oracle for their insights and feedback on my work throughout the development process. Without their help, I would not have been able to achieve the results presented in this thesis.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

List of Tables

List of Figures

List of Listings

1. Introduction

Java is one of the most widely deployed programming languages in enterprise and server-side computing, where applications routinely manage heaps of tens to hundreds of gigabytes. In these environments, the performance of the Java Virtual Machine's (JVM) garbage collector (GC) directly affects application latency and throughput. The Z Garbage Collector (ZGC) addresses this by performing nearly all garbage collection work concurrently with the application, achieving sub-millisecond pause times regardless of heap size.

However, one aspect of ZGC that has received comparatively little optimization attention is its processing of *weak references*. Java's `WeakReference` class allows applications to hold references that do not prevent the referent from being collected, and it is widely used in caching libraries, canonical mappings, and listener registrations. When the garbage collector determines that a weakly referenced object is no longer strongly reachable, it must clear the reference and if the reference was registered with a `ReferenceQueue`, enqueue it for the application to process.

The current ZGC reference processing pipeline treats all discovered weak references uniformly: they are linked together in an intrusive linked list, processed, and then handed to a Java-level daemon thread (the `ReferenceHandler`) for enqueueing. This design introduces overhead that is disproportionate for a specific subset of weak references: those that are not registered with a `ReferenceQueue`.

This thesis investigates whether targeted modifications to ZGC's reference processing pipeline can reduce this overhead. Three orthogonal optimization approaches are explored: skipping the enqueue path for references without queues, replacing the intrusive linked list with a cache-friendly dynamic array for such references, and reducing per-reference instruction cost through sequential code optimizations.

1.1 Motivation

Weak references occupy a unique position in JVM garbage collection. Unlike regular references, which are handled implicitly by the mark phase, weak references require explicit processing by the collector: each discovered weak reference must be individually evaluated, its referent cleared if unreachable, and the reference potentially enqueued. This per-reference processing cost

scales linearly with the number of weak references, making it a bottleneck in applications that allocate millions of them.

The problem is amplified by a mismatch between the API design and its typical usage. Java's `WeakReference` API provides an optional `ReferenceQueue` parameter, and the garbage collector's pipeline is designed around the assumption that cleared references will always be enqueued. The result is a pipeline that performs linked-list threading, cross-thread handoff, and iteration work for references that will never be enqueued.

ZGC's concurrent architecture makes this inefficiency particularly relevant. Because ZGC processes references concurrently with the application, any reduction in per-reference overhead translates directly into less time spent in the reference processing phase, freeing GC worker threads for other tasks.

1.2 Problem Statement

The current ZGC reference processing pipeline does not distinguish between weak references with and without reference queues during the discovery and enqueue phases. All cleared references are threaded onto a pending list, transferred to the `ReferenceHandler` thread, and iterated regardless of whether they will actually be enqueued. Additionally, the intrusive linked-list data structure used for discovered references exhibits poor cache locality and overloads the `discovered` field for both GC-internal linking and pending-list linking.

This thesis seeks to determine whether these inefficiencies can be addressed through modifications to ZGC's reference processor, and to quantify the resulting performance impact.

1.3 Research Questions

The main research questions that this thesis aims to address are:

1. What design changes and optimizations can be made to the weak reference processing in ZGC to improve its performance?
2. How do these design changes and optimizations impact the overall performance and memory footprint of ZGC?

1.4 Stretch Goals

In addition to the primary research questions, the following stretch goals are defined to explore the broader implications of this work:

1. Does the introduction of a weak field annotation in Java provide a more efficient way to handle weak references compared to the current `WeakReference` object-based implementation? A weak field would

allow individual fields of an object to have weak-reference semantics without allocating a separate `WeakReference` object, potentially eliminating both the memory overhead and the reference processing cost entirely.

2. Can the optimizations developed for ZGC be applied to other garbage collectors in the JVM, such as the G1 garbage collector, and if so, what performance improvements can be observed? The shared reference processor used by G1 has a different threading model (using CAS-based discovery), so the applicability and performance characteristics of the optimizations may differ.

1.5 Scope and Delimitations

This thesis focuses on the reference processing pipeline within ZGC, specifically targeting weak references. While the optimization approaches are described in general terms and the stretch goals consider applicability to G1 and alternative API designs, the primary implementation and evaluation are conducted within the ZGC codebase of OpenJDK.

The correctness of the optimizations is verified through the standard OpenJDK test suite and manual inspection; formal verification is outside the scope of this work. Performance measurements are conducted on a single hardware platform under controlled conditions so therefore may not generalize to all environments.

2. Background

The three optimization approaches laid out in **??** rely on several key concepts and design decisions in the Java Virtual Machine's (JVM) garbage collection architecture, the Z Garbage Collector (ZGC), and the Java reference processing pipeline. This chapter provides the necessary background on these topics to understand the motivation and implementation of the optimizations.

2.1 Garbage Collection in the JVM

The Java Virtual Machine (JVM) provides automatic memory management through garbage collection (GC). Rather than requiring the programmer to explicitly allocate and free memory as in languages such as C or C++, the JVM tracks which objects are reachable from the application's root set, comprising of thread stacks, static fields, and JNI references, and reclaims the memory of objects that are no longer reachable.

The JVM ships with several garbage collector implementations, each designed with different trade-offs between throughput, latency, and memory footprint [1]. The collectors are:

- **Serial GC** A single-threaded collector that performs all garbage collection work using one thread, avoiding inter-thread communication overhead. It is best suited to single-processor systems and small heaps.
- **Parallel GC** A throughput-oriented generational collector that uses multiple GC threads to speed up collection on multiprocessor systems. It is intended for medium to large data sets.
- **G1 GC (Garbage-First)** A mostly concurrent, region-based collector designed to scale from small to large multiprocessor machines. It targets predictable pause times with high probability while maintaining high throughput and is the default collector on most platforms.
- **ZGC (Z Garbage Collector)** A scalable, low-latency collector that performs expensive GC work concurrently with application threads. It targets pause times of only a few milliseconds (largely independent of heap size) and supports heap sizes from 8 MB to 16 TB.

This thesis focuses on ZGC, as its low-latency design and concurrent reference processing pipeline can particularly benefit from optimizations that reduce the amount of work done during reference processing.

2.1.1 Generational Collection

All JVM garbage collectors (Serial, Parallel, G1, and Z) employ *generational collection*, based on the *weak generational hypothesis*: most objects die young [2] and the ones that don't tend to live for a long time. The heap is divided into at least two generations:

- The **young generation**, where newly allocated objects reside. Young generation collections are frequent but fast, since most young objects are short-lived.
- The **old generation**, where objects that have survived multiple young collections are promoted to. Old generation collections are less frequent but more expensive since live objects are more plentiful.

ZGC adopted generational collection starting with JDK 21. In generational ZGC, the young and old generations can be collected independently and therefore concurrently. This is relevant to reference processing because references in different generations are handled during different collection cycles.

2.2 Java Reference Types

In addition to ordinary (strong) references, Java provides *reference objects* in the `java.lang.ref` package that allow applications to refernece objects without preventing their garbage collection. These reference types are:

1. **SoftReference** The referent is cleared at the discretion of the GC, typically when memory is low. Commonly used for memory-sensitive caches.
2. **WeakReference** The referent is cleared as soon as it becomes weakly reachable (i.e., no strong or soft references exist to it). Widely used in caching frameworks, canonical mappings, and listener registrations.
3. **FinalReference** An internal JVM type (not part of the public API) used to implement the `finalize()` mechanism. Objects with finalizers are wrapped in a `FinalReference` so the GC can schedule finalization before reclaiming the object. This scheduling is done via a `ReferenceQueue` which means that FinalReferences always have an associated `ReferenceQueue`.
4. **PhantomReference** The referent is never accessible through the reference; `get()` always returns `null`. Used for post-mortem cleanup actions, often to avoid the pitfalls of finalization. Phantom references must be registered with a `ReferenceQueue` to be useful.

All four types extend the abstract class `Reference<T>` (in the `java.lang.ref` package), which defines the key fields and the lifecycle managed by the garbage collector.

2.2.1 The Reference Object Lifecycle

Each `Reference` object maintains several fields that track its state:

- `referent` The referenced object. The GC may clear this field (set it to `null`) when the referent becomes eligible for collection according to the reference's type.
- `queue` The `ReferenceQueue` this reference is registered with. If the reference was not created with a queue, this field points to the sentinel object `ReferenceQueue.NULL_QUEUE`.
- `next` Used as a link pointer when the reference is enqueued in a `ReferenceQueue`.
- `discovered` A field with dual purpose: during GC marking, it serves as a link in the garbage collector's internal *discovered list*; after processing, it is reused as a link in the VM's *pending list* that feeds the `ReferenceHandler` thread. Due to its first use, the GC can detect whether a reference has already been discovered by checking if this field is `null`.

A reference object transitions through the following states during its lifetime:

1. **Active** The reference has been created and the referent is still reachable (or not yet processed).
2. **Pending** The GC has determined the referent is eligible for clearing and has added the reference to the VM's pending list. The `ReferenceHandler` thread will process it.
3. **Enqueued** The `ReferenceHandler` thread has moved the reference into its associated `ReferenceQueue`, where the application can retrieve it via `poll()` or `remove()`.
4. **Inactive** The reference has been dequeued by the application, or was created without a queue and has been cleared. It is no longer tracked by the GC or queuing infrastructure.

An important observation is that references created *without* a `ReferenceQueue` can transition directly from Active to Inactive, bypassing the Pending and Enqueued states entirely. However, as discussed in section 2.4, the default implementation does not take advantage of this shortcut.

2.2.2 WeakReference and ReferenceQueue

A `WeakReference` is the most commonly used non-strong reference type in Java applications. It has two constructors, see Listing 1:

When a `WeakReference` is created with a `ReferenceQueue`, the application can be notified when the referent is collected. When created *without* a queue, the reference serves only as a conditional pointer, the application can call `get()` to check whether the referent is still alive, but receives no notification upon clearing.

```java
// Without a queue - queue field set to NULL_QUEUE
public WeakReference(T referent) {
    super(referent);
}

// With a queue
public WeakReference(T referent, ReferenceQueue<? super
    T> q) {
    super(referent, q);
}
```

Listing 1: WeakReference constructors

The `ReferenceQueue` is implemented as a synchronized singly-linked list using the `next` field of the `Reference` objects. It provides `poll()` for non-blocking retrieval and `remove()` for blocking retrieval with an optional timeout. This is the mechanism by which applications can react to the collection of referents, for example to clean up associated resources.

A key insight for this thesis is that **many real-world uses of `WeakReference` do not use a `ReferenceQueue`**. Caching frameworks, `WeakHashMap`, and similar data structures typically create weak references without queues, relying instead on periodic polling or lazy cleanup. This means the garbage collector expends effort preparing these references for enqueueing (adding them to the pending list) even though the `ReferenceHandler` thread will immediately skip them.

2.3 The Z Garbage Collector

ZGC is a concurrent, region-based, compacting garbage collector included in the JDK since JDK 11 (experimental) and production-ready since JDK 15. Starting with JDK 21, ZGC supports generational collection, which is now the default mode. ZGC's primary design goals are:

- Sub-millisecond pause times that do not increase with heap size.
- Support for heap sizes from 8 MB to 16 TB.
- Concurrent marking, relocation, and reference processing.

ZGC achieves these goals through several key techniques:

Coloured Pointers.
ZGC embeds metadata in the unused bits of 64-bit object pointers. These *colour bits* encode information about the pointer's current state relative to the GC's phase, for example, whether the pointer has been remapped after relocation, or whether it points to a young or old generation object. This allows the GC to reason about pointer states without loading the object header.

Barriers.
ZGC uses barriers at every reference load or write. The barrier checks the colour bits of the loaded pointer and, if necessary, applies a fixup (e.g., remapping after relocation or marking for concurrent marking). This is the mechanism that allows ZGC to perform most of its work concurrently with the application.

Concurrent Phases.
A ZGC collection cycle interleaves short stop-the-world pauses (for root scanning and synchronization) with concurrent phases (for marking, reference processing, and relocation). The concurrent phases run alongside application threads, using barriers to maintain consistency.

Multiple GC Worker Threads.
ZGC uses multiple GC worker threads to parallelize GC work. Each worker thread maintains its own data structures in many cases to avoid contention.

2.4 Reference Processing Pipeline in ZGC

Reference processing is the mechanism by which the garbage collector handles `Reference` objects discovered during marking. The pipeline has three stages: *discovery*, *processing*, and *enqueueing*. In ZGC, the first two stages run concurrently with the application during the marking phase, while the enqueue stage involves handing off references to the Java-level `ReferenceHandler` thread.

2.4.1 Discovery

During the concurrent marking phase, when the GC encounters a `Reference` object (i.e., an instance of `SoftReference`, `WeakReference`, `FinalReference`, or `PhantomReference`), it evaluates whether the reference should be *discovered*, i.e. added to a list for later processing.

The discovery decision depends on several factors:

1. **Is the reference active?** Only active references are eligible. For non-`FinalReference` types, an inactive reference has a `null` referent. For `FinalReference`, inactivity is indicated by a non-null `next` field.
2. **Is the referent still strongly reachable?** If the referent is already marked as strongly live, the reference does not need processing. The garbage collector checks this via its liveness information.
3. **Soft reference policy.** For `SoftReference` objects, the collector applies a policy (based on available memory and the reference's last access time) to decide whether to keep the referent alive or clear it.

4. **Generational considerations.** In ZGC, only references residing in the old generation are discovered; young-generation references are always treated as strong. This is because young-generation collections are short and the overhead of discovering and processing references would be disproportionate to the benefits, given that most young objects die quickly.

Once a reference is selected for discovery, it is recorded on a *discovered list*. In ZGC each GC worker thread maintains its own discovered-list head, so discovery is implemented as a cheap, non-atomic prepend to a per-worker list. This avoids CAS contention found in shared processors where multiple threads update the same discovered head.

2.4.2 Processing

After marking completes, the discovered lists are processed. For each discovered reference, the collector determines the final fate of the referent:

1. If the referent has become `null` (cleared by the application concurrently), the reference is dropped from the list.
2. If the referent is now strongly reachable (e.g. it was marked as alive after the reference's discovery), the reference is dropped and the referent is kept alive.
3. If the referent is still unreachable according to the reference's strength, the referent is **cleared** (set to `null`), and the reference is prepared for enqueueing.

In ZGC, clearing involves setting the `referent` field to a coloured null pointer via compare-and-swap (CAS) to ensure thread safety since application threads may be concurrently accessing the reference.

After clearing, the reference is linked onto the pending list for the `ReferenceHandler` thread. This involves writing the reference's address into the `discovered` field of the previously processed reference and atomically prepending it to the GC's internal pending list.

Generational interactions are important during processing: when a `Reference` object is old but its referent is young, the referent is treated as strongly reachable for the duration of the old-collection and must not be cleared. Processing must therefore verify generation membership and, if appropriate, ensure young-generation liveness is preserved (by marking the young referent) so it is not lost between concurrent phases.

2.4.3 Enqueue

After a reference's referent has been cleared, the reference must be made available to the application. This happens through the *pending list* mechanism:

1. The GC builds an internal pending list by linking cleared references together via their `discovered` field.

2. Under the heap lock, the GC atomically prepends its internal pending list onto the global VM pending list.
3. The `ReferenceHandler` daemon thread is notified.
4. The `ReferenceHandler` thread wakes up, atomically retrieves the pending list, and iterates over it. For each reference, it checks whether `queue != NULL_QUEUE`. If the reference has a real queue, it calls `queue.enqueue(this)`; otherwise it simply advances to the next reference.

This handoff reveals an inefficiency: references created without a `ReferenceQueue` are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be skipped. For workloads where the majority of weak references lack queues, this constitutes unnecessary bookkeeping. This behavior has been reported in an OpenJDK enhancement issue[3]. Avoiding enqueue-path work for references with `NULL_QUEUE` is one of the primary optimization target explored in this thesis.

3. Method

In order to answer the research questions posed in section 1.3 three orthgonal approaches were implemented to optimize the reference processing of weak references in Z garbage collector (ZGC). These approaches were implemented in ZGC in different combinations and their performance was evaluated using microbenchmarks. These three approaches were as stated in chapter 1 the following:

- Skipping the enqueue path for references without a `ReferenceQueue`
- Replacing the intrusive linked list with a dynamic array for the discovered list for references without a `ReferenceQueue`
- Sequential Code Optimizations

The rationale and implementation details of these approaches are described in the following sections. The evaluation of the approaches is described in chapter 4.

3.1 Skipping Enqueue Path

As described in section 2.4.2 ZGC enqueues discovered references to the internal pending list regardless of whether they have a `ReferenceQueue` or not. This is inefficient for weak references without a `ReferenceQueue` as discussed in section 2.4.3. Because of this, the first optimization approach implemented was to skip the enqueue path for references without a `ReferenceQueue`. The intended effect of this optimization is to both reduce the amount of work done by the GC threads during reference processing and to reduce the amount of work done by the `ReferenceHandler` thread when iterating over the pending list.

3.1.1 Implementation of Null-queue Fast Path

In order to skip enqueueing references without a `ReferenceQueue` the `ReferenceProcssor` class was modified to check if the queue field of the reference being discovered is equal to the sentinel value `ReferenceQueue.NULL_QUEUE` before adding it to the discovered list. If the reference does not have a `ReferenceQueue` it is added to a separate discovered list for references without a `ReferenceQueue` instead of being added to the regular discovered list. This separate discovered list is then processed separately by the GC threads and none of its references are enqueued to the pending list for the `ReferenceHandler` thread. This optimization is controlled by a JVM flag and can be enabled or disabled at runtime.

4. Results

5. Discussion

6. Conclusion

References

[1] Oracle, “Available collectors.” https://docs.oracle.com/en/java/javase/25/gctuning/available-collectors.html, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-02-24.

[2] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981. Accessed: 2026-02-24.

[3] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.” https://bugs.openjdk.org/browse/JDK-8029205, n.d. Accessed: 2026-02-24.